中图分类号：TN492

学号：ZY2402113

北京航空航天大學

# 硕士中期检查报告

# 基于RISC-V的
# 软硬件协同及编译

作者姓名 蒋一平

学科专业 电子信息

指导教师 李洪革教授，李丽副教授

培养院系 电子信息工程学院

2026年10月

# 目录

1.开题报告的论文研究方案 ..... 1
1.1 研究背景与意义 ..... 1
1.2 主要研究内容 ..... 1
1.3 关键技术与预期新意 ..... 2
2.开题报告的论文工作计划 ..... 3
3.已研究的内容和实现技术及关键技术的突破 ..... 3
3.1 Linux系统相关 ..... 3
3.2 AI编译器相关 ..... 4
3.2.1 总体设计与部署方式 ..... 4
3.2.2 模型解析与参数整理 ..... 6
3.2.3 内存规划 ..... 7
3.2.4 指令生成与文件导出 ..... 8
3.2.5 系统验证与阶段成果 ..... 9
4.已研究工作总结 ..... 10
4.1 Linux系统相关 ..... 10
4.2 AI编译器相关 ..... 11
5.后续研究的内容和实现技术 ..... 11
5.1 面向布局约束的生命周期与存储复用 ..... 11
5.2 分支调度与存储布局的联合优化 ..... 12
5.3 面向宏指令NPU的后处理融合与能力扩展 ..... 13
5.4 数值验证、代价统计与实验设计 ..... 13
5.5 后续工作安排与预期成果 ..... 15
6.已发表的论文和取得的成果 ..... 15
7.参考文献 ..... 16

# 1.开题报告的论文研究方案

## 1.1 研究背景与意义

近年来，随着嵌入式系统与人工智能应用的发展，端侧设备对计算性能、资源利用效率和系统可定制性的要求不断提高。RISC-V指令集具有开放、模块化和可扩展等特点，为构建自主可控的处理器及片上系统提供了基础。基于FPGA构建集成RISC-V处理器、显示模块和NPU的SoC，能够为软硬件协同设计提供可重构的实验平台。然而，硬件功能的实现还需要操作系统、设备驱动和编译工具的配套支持。本课题以“基于RISC-V的软硬件协同及编译”为研究主题，围绕Linux系统移植及驱动开发、面向NPU的AI编译器构建与优化两个方面开展研究，以完善自研SoC的软件支持并提升其面向实际应用的运行能力。

在操作系统方面，Linux具有开源、模块化和可移植性较强等特点，能够为应用程序提供进程管理、存储管理和设备访问等基础服务，是完善RISC-V软件环境的重要组成部分。针对FPGA平台资源有限、处理器运行频率受约束以及自研外设缺少配套驱动等问题，需要结合目标SoC设计轻量级引导程序，完成Linux内核、设备树和根文件系统的适配与裁剪，并为图像显示模块、NPU等外设开发相应驱动。通过对启动流程、系统组成与外设访问方式进行协同设计，可以在满足应用功能的基础上控制系统体积和启动开销，为图形交互与异构计算应用提供统一、可靠的软件运行环境。

在AI编译器方面，神经网络通常以计算图和张量运算描述，而实验室自研NPU采用具有特定编码、数据格式和地址要求的宏指令，两者之间存在模型语义、物理布局和执行控制等方面的差异。AI编译器需要通过计算图解析、算子映射、静态存储规划与指令生成，将模型转换为目标NPU能够执行的程序。已有研究表明，图级优化与算子级映射共同影响模型在目标平台上的执行效率[1]。因此，面向自研NPU构建专用离线编译器，不仅能够建立模型到硬件执行的自动转换流程，还能够结合宏指令粒度、通道对齐及张量依赖等约束，研究执行调度与存储布局的优化方法，为发挥专用加速器的计算能力提供编译支持。

## 1.2 主要研究内容

本课题的研究内容主要包括以下两个方面。

（1）RISC-VSoC的Linux系统移植及驱动开发。以FPGA实现的RISC-VSoC为基础，研究系统启动、内核适配、根文件系统构建以及外设访问所需的软件支持，建立处理器与片上外设之间的基本软件接口。该部分工作关注操作系统与硬件平台之间的适配关系，为后续系统应用提供支撑。

（2）面向NPU的AI编译器构建与优化。围绕神经网络等AI模型从模型描述到NPU执行的转换过程，研究模型导入、中间表示、算子合法化、张量布局、参数组织、存储地址规划以及指令生成等关键问题。在完成离线编译与板端部署的基础上，进一步分析宏指令执行方式和多分支网络的数据依赖，探索降低中间张量存储占用及冗余数据搬运的方法。

编译部分采用部署前完成主要决策的技术路线。对于结构和输入形状确定的模型，在PC侧生成固定执行计划、指令序列及参数数据，设备侧负责加载编译产物、配置执行环境、启动NPU并获取结果。TVM等研究说明了将模型级优化与硬件相关代码生成相结合的可行性[2]；本课题根据自研NPU的指令和存储特征，构建范围明确、能够直接服务于原型系统的编译流程。

## 1.3 关键技术与预期新意

Linux系统方面，关键技术包括面向自研RISC-V SoC的启动环境构建、精简SBI服务实现、Linux内核与根文件系统裁剪，以及设备树适配和定制外设驱动开发。结合开题报告中的研究方案，预期新意在于围绕FPGA平台的资源与频率约束，对引导程序、操作系统配置和外设访问进行协同设计：自主开发仅包含必要硬件初始化、内核加载及启动支持的轻量级引导程序，并结合目标应用裁剪内核与根文件系统，在功能完整性、系统体积和启动效率之间进行权衡；同时，针对图像显示模块建立相应驱动接口，结合硬件条件组织中断响应和数据传输，支撑处理器、显示模块的协同运行，形成面向本SoC的定制化Linux软件支持方案。

AI编译器方面，关键技术与预期新意主要包括两个方面。一是面向实验室自研、采用宏指令体系的NPU构建专用离线编译器，针对单条指令执行较大粒度张量操作的特点，研究计算图到宏指令执行序列的映射方法，将算子分解、通道分组、参数组织及静态地址规划相结合，在PC侧生成与目标硬件匹配的指令和参数二进制文件，建立从模型描述到RISC-VSoC中NPU执行的编译路径。二是在已有编译流程基础上，研究面向宏指令及物理布局约束的执行调度与存储联合优化，将张量生

命周期、分支依赖、通道对齐和拼接别名关系纳入统一分析，联合选择合法执行顺序与缓冲区复用方案，降低中间张量的峰值存储需求，并减少具备消除条件的数据搬运。前者体现面向自研硬件的专用编译适配，后者作为后续重点研究的优化方法，通过同一硬件平台上的对照与消融实验评价其有效性。

## 2.开题报告的论文工作计划

开题阶段的总体工作安排按照系统构建、软件适配、编译优化与论文整理依次展开。结合目前已经具备的NPU编译和FPGA验证条件，在保持系统软件与编译两个研究方向的基础上，将编译部分的工作进一步细化为流程完善、约束分析、优化算法实现和板端实验。原计划与当前阶段安排如表1所示。

表1 论文研究工作计划

| 时间安排 | 计划内容 | 阶段说明 |
| --- | --- | --- |
| 2025.12—2026.02 | 构建RISC-VSoC与FPGA验证环境，集成相关外设 | 原计划中的硬件平台阶段 |
| 2026.03—2026.04 | 研究启动加载与基础软件适配 | 系统软件方向 |
| 2026.05—2026.07 | Linux内核与根文件系统移植 | 具体进展在Linux章节补充 |
| 2026.08—2026.09 | 外设驱动开发与系统联调 | 具体进展在Linux章节补充 |
| 2026.10—2026.12 | 面向NPU的AI编译与优化研究 | 已有全流程基线，继续开展方法与实验 |
| 2027.01—2027.04 | 补充实验、撰写与修改学位论文 | 优化验证与论文整理交叉进行 |
| 2027.05 | 论文答辩 | 按学院实际安排执行 |

## 3.已研究的内容和实现技术及关键技术的突破

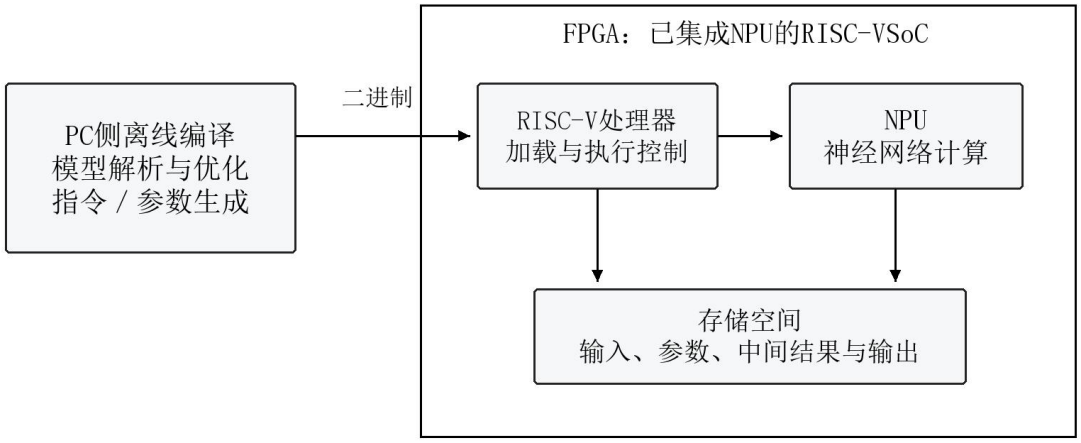
### 3.1 Linux系统相关

## 3.2 AI编译器相关

### 3.2.1 总体设计与部署方式

本课题围绕实验室自研NPU构建离线AI编译器，将神经网络模型转换为目标硬件能够直接执行的指令和参数文件。目前已完成模型解析、参数整理、内存规划、指令生成及文件导出等主要模块，并在集成NPU的RISC-VSoC上完成FPGA验证，能够运行编译器生成的YOLO网络二进制文件并正确进行图像识别。编译器的设计目标是将原先需要人工完成的模型信息提取、数据排列、地址计算和指令配置组织为统一流程，为自研NPU的模型部署与后续优化提供软件基础。

编译系统采用PC侧编译、SoC侧加载执行的部署方式，如图1所示。在PC侧，编译器依据模型结构和目标硬件约束，预先确定运算顺序、参数布局及中间结果地址，生成完整的NPU指令流。部署时，将指令、参数和输入数据装载到SoC的指定存储区域，由RISC-V处理器完成必要的地址配置与启动控制，随后NPU按指令流读取数据、执行计算并写回结果。端侧不需要重新解析模型计算图，加载与控制程序主要负责连接编译产物和硬件执行接口。




系统结构示意；实际总线与加载接口以平台实现为准

图1 离线AI编译器与RISC-V异构SoC的关系

编译器总体流程如图2所示，主要由模型前端、内存规划、算子指令生成和文件导出等部分组成。前端将模型结构转换为统一的中间表示（IR），并提取权重、偏置等参数；内存规划模块结合硬件约束确定计算过程及数据地址；后端据此生成汇编指令，经汇编编码后形成机器码，最终输出部署所需文件。各阶段通过模型结构文件和内存规划文件传递信息，使模型解析与硬件指令生成保持相对独立。

图2给出了编译流程的各阶段及主要产物。其中，PT2是当前已实现的模型输入路径，ONNX表示可通过新增前端进行适配的扩展方向。图中的“NPU可执行文件”指目标NPU可加载的指令及参数产物；在BRAM初始化场景下可按布局合并组织，在DDR部署场景下则分别导出指令和参数二进制文件，以适配不同的装载地址。

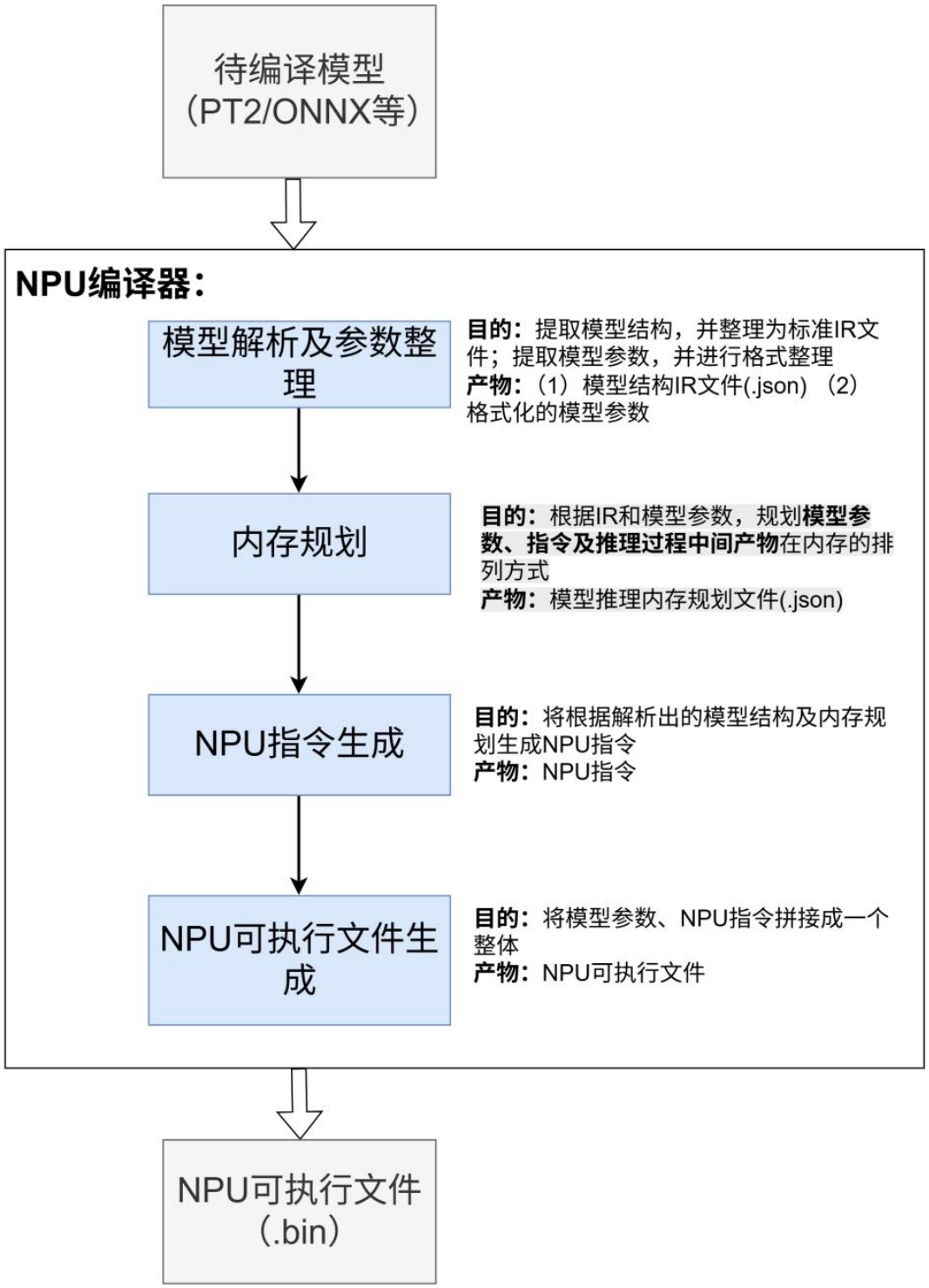



图2 NPU编译器总体流程

### 3.2.2 模型解析与参数整理

模型解析的主要任务是从原始模型中提取计算结构与参数，并转换为后续模块能够统一处理的形式。目前前端以PyTorch导出的PT2模型为输入，读取计算节点、张量形状及参数映射关系，将算子类别、输入输出连接、卷积核大小、步长和填充等信息写入图中间表示，同时保存模型权重与偏置。统一的IR使后端能够依据模型语义完成处理，而不必在每个编译阶段重复读取原始模型。该分层方式与通过中间表示逐级完成模型到目标代码转换的思路一致[3]。

以连续的卷积和ReLU运算为例，前端首先记录两个节点的运算属性及数据依赖，明确ReLU的输入来自卷积输出；随后提取卷积所需的权重和偏置，供参数生成与地址规划使用。现有前端已建立卷积、激活、池化、最近邻上采样、填充、通道拼接和全连接等操作到内部表示的映射，具体可执行的参数范围由后续硬件适配规则进一步约束。若需要支持其他模型格式，可增加相应解析入口，将其转换为相同的IR，并完成算子语义和数值行为的校验后复用后端流程。

模型参数还需按照NPU的数据读取方式进行整理。框架导出的权重排列与硬件使用的通道分组、像素交织顺序可能不同，因此参数生成模块需要完成权重重排、必要的通道补齐及偏置组织。对于采用4通道对齐的情况，编译器以通道组为单位安排存储，将不足一组的部分补齐，使指令所描述的通道数量与实际参数空间相匹配。图3给出了逻辑通道与物理存储之间的关系。

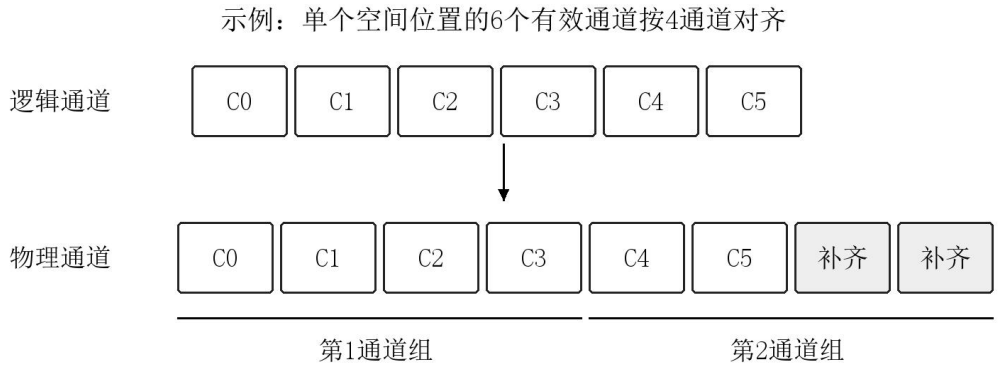



图3 通道对齐对物理存储空间的影响示意

参数整理与执行计划共同决定硬件实际读取的数据。例如，输出通道较多的卷积会根据目标硬件能力拆分为多个计算分组，每个分组需要对应的权重范围和输出区域。编译器依据分组结果生成参数数据及偏移信息，保证后续指令能够访问正确

的参数。对于具有偏置和不具有偏置的操作，也分别组织相应内容，避免参数长度或排列方式与指令配置不一致。

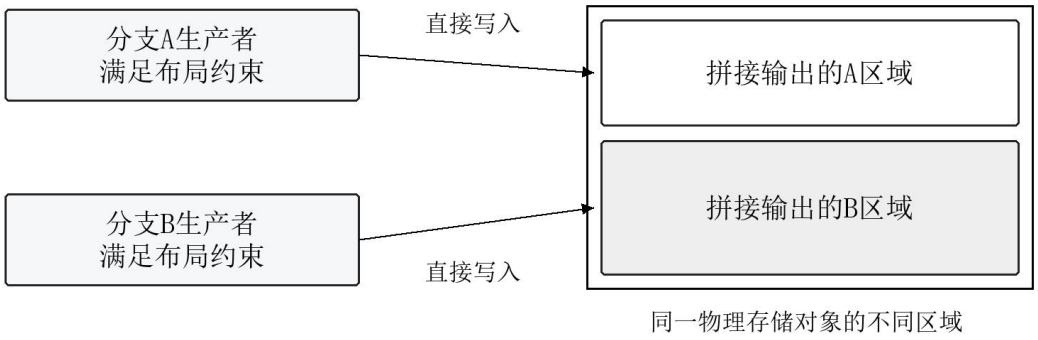
### 3.2.3 内存规划

内存规划是连接模型描述与指令生成的关键环节。该阶段以统一IR和硬件配置为依据，确定输入数据、模型参数、中间张量、模型输出及指令所需的存储空间，并形成memory_plan.json等规划文件。规划结果不仅记录各区域的起始地址和大小，还包含算子分组、参数偏移及输出位置等执行信息，供参数生成、指令生成与文件导出共同使用。通过在编译阶段完成这些工作，可以减少人工计算地址的工作量，并保持各阶段对同一数据的解释一致。

仍以卷积后接ReLU为例，规划器首先根据输入形状、卷积属性和通道对齐规则计算卷积输出的物理大小，为其分配存储区域；随后将该输出地址作为ReLU的输入地址，并为激活结果安排输出位置。与只按逻辑张量元素数量估计空间相比，实际地址计算还需要考虑对齐填充、分组方式和硬件寻址规则。规划完成后，编译器检查相关地址区间的边界及重叠情况，后端据此生成一致的寄存器配置。

针对模型运算规模与NPU单次计算能力不一致的情况，内存规划同时组织相应的执行步骤。例如，卷积输出通道可按硬件支持的粒度进行分组，逻辑上的一次卷积由多次计算完成，各组分别使用对应的参数和输出区域。对于部分需要组合实现的运算，则依据目标指令能力安排相邻操作及数据连接。这一过程将计算图中的逻辑节点转换为可以实际执行的操作序列，使后续指令生成集中于具体字段的配置和编码。

对于满足条件的通道拼接，当前编译器采用预先安排生产者输出位置的方法，使各分支直接写入最终拼接张量的对应区域，如图4所示。当拼接方向、空间尺寸及通道对齐满足目标布局要求时，后续算子可以直接读取该区域，从而省去单独的拼接数据拷贝。这一处理将网络中的连接关系与物理存储布局结合起来，减少了部分不必要的数据搬运。




无额外拼接拷贝；复用时必须共同检查别名与消费者

图4 满足布局条件时的通道拼接免拷贝示意

目前中间结果区域仍以顺序分配为主，尚未实现面向全图生命周期的通用缓冲区复用。随着网络深度和分支数量增加，已完成消费的中间张量仍可能占用原有空间。同时，拼接预布局会使多个逻辑张量共享同一物理存储对象的不同区域，复用时需要统一处理其别名和使用关系。因此，现有规划文件也为后续开展生命周期分析、分支调度与存储联合优化提供了基础。

### 3.2.4 指令生成与文件导出

在模型执行顺序、数据布局和存储地址确定后，算子编译模块读取内存规划文件，为各执行节点生成对应的汇编片段。目标NPU采用宏指令形式，通常先通过配置指令设置输入地址、输出地址、参数地址和算子属性，再由计算指令触发相应硬件单元。编译器因此需要在指令生成前完成尺寸、通道分组和地址等静态计算，使硬件能够依据预先生成的配置执行运算。

以卷积运算为例，后端先设置输入、输出和权重地址所在的通用寄存器，再配置卷积核大小、通道数量及偏置等相关字段，最后生成CONV指令启动计算；后续ReLU节点则引用卷积结果地址并生成相应配置与计算指令。图5给出了卷积与ReLU的汇编示例，展示了地址配置、算子属性配置和计算启动之间的关系。图中地址用于说明该示例的编译结果，实际模型的地址由对应内存规划决定。

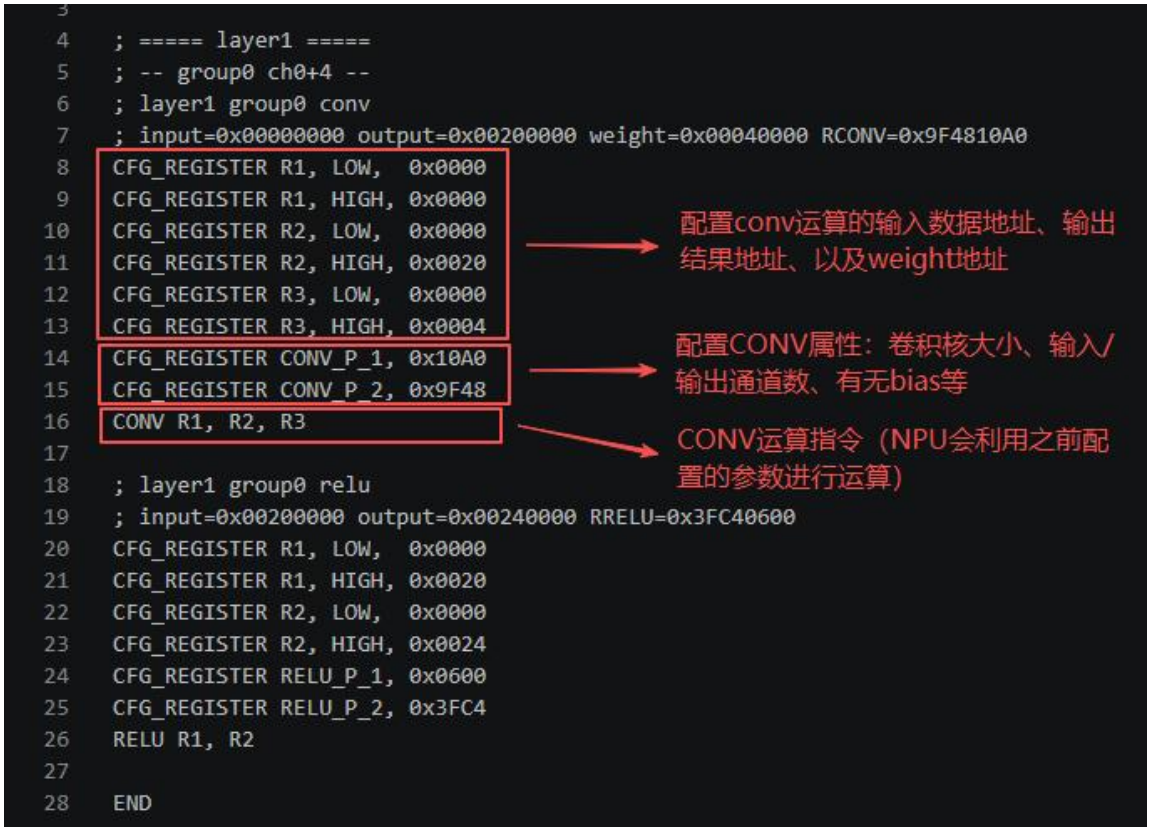



图5 卷积与ReLU的汇编指令示例

汇编器按照目标指令编码规则，将上述汇编序列转换为32位机器码。构建脚本进一步组织参数与指令导出：对于BRAM仿真和初始化场景，可生成按预定布局组织的COE等文件；对于DDR部署场景，则分别输出npu_instr.bin与npu_params.bin，供系统按指定基地址加载。指令编码、参数排列和装载位置均与内存规划保持对应，构成从模型到硬件执行文件的完整转换过程。

分层的后端结构也为增加硬件算子提供了接口。新增算子时，需要在IR中补充其输入输出和属性，在规划阶段确定形状、空间需求及合法参数范围，再增加相应汇编生成规则；若引入新的指令编码，还需同步扩展汇编器。由此可在保留整体流程的基础上开展软硬件协同扩展，并通过单算子与完整网络验证检查各阶段的一致性。

### 3.2.5 系统验证与阶段成果

本阶段已将编译器与实际硬件执行过程贯通。NPU已集成到RISC-VSoC中，并随整个SoC部署到FPGA；系统能够加载本编译器生成的YOLO网络二进制文件，由NPU执行网络计算并正确完成图像识别。这一结果表明，模型解析、参数整理、内存规划、指令编码及板端装载之间已建立可运行的完整链路，为后续在同一硬件平台上比较不同编译策略提供了实验基础。

目前已确认的是完整系统中的功能验证结果。为进一步形成可重复、可比较的实验记录，后续将补充板卡型号、工作频率、模型版本、输入尺寸及量化配置等条件，并记录NPU执行周期和端到端处理时间。现阶段尚未形成统一的帧率、功耗、加速比或数据集精度统计，相关信息按表2继续整理。

表2 FPGA验证记录与待补充实验信息

| 项目 | 当前状态 | 需归档的证据 |
| --- | --- | --- |
| 系统集成与运行 | 已完成RISC-VSoC集成NPU及FPGA运行 | 系统版本、板卡照片、运行日志 |
| YOLO图像识别 | 已运行编译器生成的二进制并正确识别 | 真实输入与输出图像、模型及样本说明 |
| 硬件与模型配置 | 待补充完整记录 | FPGA型号、频率、模型版本、输入尺寸、量化格式 |
| 数值一致性 | 待补充系统性测试结果 | 软件参考、逐层输出、误差指标 |
| 存储与性能数据 | 待测量并汇总 | 工作区峰值、执行周期、计时边界及重复次数 |

图6待插入：RISC-VSoC的FPGA实物与YOLO运行结果。左侧放置FPGA开发板实物照片，标注板卡型号及图像输入、结果输出接口；右侧放置同次运行的输入图像与真实识别结果，并注明模型名称、输入尺寸和测试日期。可在下方补充指令、参数装载及执行完成的真实日志。

图6 FPGA系统运行与YOLO图像识别结果（实测材料待补充）

通过上述工作，已解决自研NPU部署过程中模型信息、数据布局和指令格式之间的转换问题，形成可重复构建的离线编译流程。后续研究将在此基础上完善逐层数值比对与性能测量，重点分析中间张量的生命周期、分支执行次序和布局约束对存储需求的影响，进一步实现执行调度与存储联合优化，并通过对照实验评价其效果。

# 4.已研究工作总结

## 4.1 Linux系统相关

### 4.2 AI编译器相关

按照编译部分的阶段目标，目前已经完成从模型导入到NPU二进制文件生成的主要软件流程，形成模型图表示、执行计划、存储计划、参数转换和指令生成等模块，并完成在RISC-V SoC集成环境下的FPGA部署验证。编译器生成的YOLO二进制文件已经能够在该系统上运行并正确进行图像识别，表明模型到硬件执行的端到端路径已经跑通。

与开题阶段的总体方案相比，目前编译研究已结合具体NPU实现进一步明确了目标平台、输入模型和输出形式，将重点落实到静态推理的图分析、数据布局和专用指令生成。研究主线仍然是RISC-V平台上的软硬件协同与编译，后续需要在现有工程基础上继续提炼优化方法。

本课题的进一步研究应针对宏指令NPU的布局和执行约束，明确现有通用方法直接应用时遇到的问题，并通过算法设计与对照实验体现改进价值。下一阶段将优先围绕存储复用、分支调度和拼接别名的共同约束开展研究。

## 5.后续研究的内容和实现技术

### 5.1 面向布局约束的生命周期与存储复用

后续首先研究静态图中张量的产生、使用和释放关系，在现有存储计划生成之前增加生命周期分析。对每个中间张量记录生产节点、全部消费者、物理尺寸及对齐要求，并在确定执行顺序后计算有效存活区间。对于已经没有后续使用的张量，其空间可以提供给后续结果复用。外部输入、模型输出、长期参数与临时激活应分别管理，避免将需要保留的数据错误回收。

基础实现将采用按执行顺序释放空间并选择可容纳区域的静态分配方法，同时保留现有顺序分配作为对照。随后针对本平台的4通道布局与拼接预布局建立物理存储对象，将共享区域的逻辑张量作为视图管理。在保守实现中，只有全部相关视图完成消费后才允许回收底层空间；进一步的子区域复用则需要证明各读写范围不重叠，并建立相应合法性检查。

研究重点不只是添加一个通用内存池，而是处理布局、别名和目标指令访问范围共同造成的约束。例如，某个分支张量已被直接消费者使用完毕，但其存储仍属于尚未完成消费的拼接输出，此时不能仅依据该分支的逻辑生命周期释放空间。拟

通过统一的存储对象描述将此类约束显式化，并比较忽略别名的简单方法、保守别名方法与联合规划方法在正确性及占用方面的差异。

评价指标包括峰值运行区大小、对齐填充量、空间碎片以及编译耗时。峰值存储需求下降并不必然带来推理加速，因此将存储收益与时间收益分别报告。若存储复用使某些原本超出地址预算的网络能够部署，也将以固定预算下的可部署性作为独立结果，而不使用未经测量的性能推断。

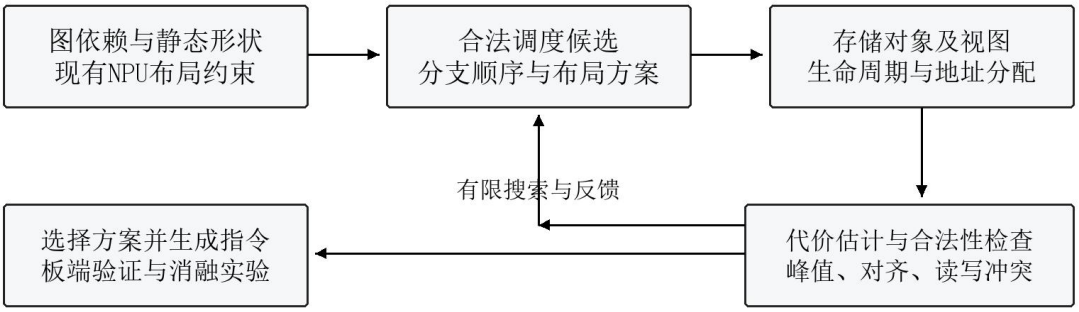
## 5.2 分支调度与存储布局的联合优化

具有多分支和拼接结构的网络，在满足依赖关系的前提下可能存在多种合法执行顺序。不同顺序会改变共享输入与分支输出的保留时间，从而影响峰值存储。MAGIS等研究已说明图变换与执行调度之间存在共同优化空间[6]。针对本课题，拟在既有拼接免拷贝处理基础上，将合法拓扑顺序、张量生命周期和物理布局共同纳入决策，而不是固定顺序后仅分配地址。

具体实现首先从当前可执行节点集合中选择候选操作，估计执行后新产生的数据、能够释放的数据及对后续汇合节点的影响，形成面向峰值占用的启发式评分。对于规模较小的分支子图，可以采用有限宽度搜索或枚举获得对照结果；对于完整网络，则限制候选数量和搜索时间，避免编译开销过大。每个候选方案均需通过拓扑合法性和存储访问冲突检查。

联合优化还需处理局部最优与全局效果之间的关系。先完成某一短分支可能暂时释放部分输入，但其输出需要等待另一长分支，从而延长存活时间；反之，优先执行长分支也可能增加中间峰值。因此，候选评分除当前释放量外，还应考虑汇合距离和输出保留需求。研究将通过典型分支子图解释算法行为，并通过完整YOLO网络验证其实际影响。

拟设置固定顺序加顺序分配、固定顺序加生命周期复用、仅调整分支顺序以及联合调度与布局规划等对照。通过消融实验分析收益分别来自空间复用、顺序改变还是布局兼容处理，避免把多个机制合并后仅给出总结果。该方向预期形成可独立描述的算法、合法性条件和实验结论，是后续论文工作的主要研究内容。




后续拟实现方法；当前已完成基线编译与FPGA功能验证

图7 后续联合调度与存储规划方法的技术路线

## 5.3 面向宏指令NPU的后处理融合与能力扩展

在完成软件优化基线后，将依据指令统计和访存分析判断是否需要扩展NPU能力。DNNFusion等工作表明融合可以减少中间结果和执行开销，但其有效性取决于目标硬件的实现方式[7]。本课题优先考虑出现频繁且语义明确的模式，例如在卷积输出阶段结合激活处理，以减少单独激活指令及中间结果的读写。是否实施该扩展将由实际瓶颈和硬件修改成本决定。

实现时需要与硬件设计协同确定激活模式、输出位宽、量化移位及饱和顺序等语义，并在指令中提供明确配置。编译器根据子图模式和数值条件选择融合路径，不满足条件时保留原有执行方式。若一个卷积输出同时被多个消费者使用，或者某个消费者需要激活前数据，则不能直接用融合结果替代全部用途，必须分析依赖与数据保留要求。

硬件比较将同时记录资源占用、最高可工作频率及推理周期，区分指令减少、访存减少和时钟变化带来的影响。DeFiNES的分析说明，跨层执行需要同时考虑不同存储层次的访问成本[8]。在本平台缓存容量、片上缓冲形式和DMA能力确认之前，不预设双缓冲或传输计算重叠一定可用；这些能力只在接口条件明确后作为补充方案进行评估。

## 5.4 数值验证、代价统计与实验设计

正确性验证将从现有YOLO端到端结果扩展到分层验证。首先整理硬件整数或定点运算约定，建立与权重、偏置、累加、移位和饱和行为一致的软件参考。量化推理研究表明，数值格式与量化位置是整数计算正确执行的重要组成部分[9]。在此

基础上，对卷积、池化、激活及典型组合子图逐层比对，再验证完整网络的输出与任务表现。

测试用例将覆盖奇数空间尺寸、非4倍数通道、多消费者张量、连续拼接以及数值边界等情况。对于仅改变地址与执行顺序的优化，原则上要求与同一硬件基线保持相同结果；对于改变数值计算顺序的融合，需预先定义可接受误差及任务指标约束。正确性检查与性能测试使用相同的模型配置，并记录编译产物和硬件版本，减少版本变化对结论的干扰。

静态分析方面，将从存储计划和指令序列统计工作区峰值、参数大小、指令条数及可计算的数据读写量。动态测量方面，将分别记录NPU执行周期和包含加载、预处理、后处理的端到端时间，明确计时边界。若硬件提供周期计数或事件计数接口，优先使用该接口；否则使用可校准的板端计时方式。所有实验保持时钟、输入和测量流程一致，并报告重复测量次数及波动范围。

代表性负载拟包含单链卷积、分支拼接子图以及当前已运行的YOLO网络，在兼容范围内改变输入尺寸和通道配置，以检查方法对不同图结构的适用性。跨编译器比较仅在目标硬件和算子支持具有可比性时开展，主要采用同一后端上的优化开关进行对照。表3列出了实验组与主要评价内容。

表3 编译优化对照实验设计

| 实验组 | 配置 | 主要比较目的 |
| --- | --- | --- |
| 基线组 | 固定顺序＋现有顺序分配 | 建立功能、存储与周期基线 |
| 生命周期组 | 固定顺序＋别名安全的空间复用 | 分析静态复用带来的容量收益 |
| 调度组 | 改变合法分支顺序，保持统一分配策略 | 分析调度对生命周期和峰值的影响 |
| 联合优化组 | 分支调度＋布局约束＋空间复用 | 评价联合决策相对独立阶段的收益 |
| 融合扩展组（条件性） | 加入经验证的硬件后处理融合 | 比较访存、周期、频率与硬件资源 |
| 共同验证 | 相同模型、输入、频率和数值约定 | 逐层正确性、任务表现与可复现性 |

## 5.5 后续工作安排与预期成果

后续研究按照“基线记录—方法实现—对照验证—论文整理”的顺序推进。首先补齐硬件能力说明、YOLO配置及板端证据，建立可重复执行的测试流程；其次实现生命周期分析与别名约束下的静态复用，形成稳定的存储优化基线；随后开展分支调度与布局联合优化，并进行消融实验；最后根据测量结果决定是否加入小范围硬件扩展，完成论文分析与材料整理。具体安排如表4所示。

表4 中期检查后的研究安排

| 时间 | 主要任务 | 阶段产物 |
| --- | --- | --- |
| 2026.10—2026.11 | 整理硬件约束与基线；实现生命周期和别名分析 | 能力说明、数值测试、存储分析 |
| 2026.12—2027.01 | 实现布局约束下的复用及分支联合调度 | 算法原型、合法性检查、对照结果 |
| 2027.02—2027.03 | 完成多负载测试及消融；按条件评估硬件扩展 | FPGA实测数据与适用范围分析 |
| 2027.03—2027.04 | 整理实验，完成论文写作与修改 | 论文初稿、修改稿与答辩材料 |
| 2027.05 | 完成答辩及材料归档 | 按学院安排完成提交 |

预期形成的成果包括一套能够稳定生成目标NPU指令与参数的离线编译工具链、一项面向通道布局与别名约束的联合调度和存储优化方法，以及相应的正确性测试和FPGA实验结果。若硬件扩展验证能够在既定时间内完成，则补充编译映射与硬件代价分析。论文将以实际实验结果为依据说明收益及适用范围，不预设未经验证的加速比例或存储降低幅度。

## 6.已发表的论文和取得的成果

无。

# 7.参考文献

[1] Li M, Liu Y, Liu X, et al. The Deep Learning Compiler: A Comprehensive Survey[J]. IEEE Transactions on Parallel and Distributed Systems, 2021, 32(3): 708-727. DOI: 10.1109/TPDS.2020.3030548.

[2] Chen T, Moreau T, Jiang Z, et al. TVM: An Automated End-to-End Optimizing Compiler for Deep Learning[A]. In: 13th USENIX Symposium on Operating Systems Design and Implementation[C]. Berkeley: USENIX Association, 2018: 578-594. https://www.usenix.org/conference/osdi18/presentation/chen.

[3] Lattner C, Amini M, Bondhugula U, et al. MLIR: Scaling Compiler Infrastructure for Domain Specific Computation[A]. In: 2021 IEEE/ACM International Symposium on Code Generation and Optimization[C]. Piscataway: IEEE, 2021: 2-14. DOI: 10.1109/CGO51591.2021.9370308.

[4] Burrello A, Garofalo A, Bruschi N, et al. DORY: Automatic End-to-End Deployment of Real-World DNNs on Low-Cost IoT MCUs[J]. IEEE Transactions on Computers, 2021, 70(8): 1253-1268. DOI: 10.1109/TC.2021.3066883.

[5] Lin J, Chen W M, Lin Y, et al. MCUNet: Tiny Deep Learning on IoT Devices[A]. In: Advances in Neural Information Processing Systems 33[C]. Red Hook: Curran Associates, 2020: 11711-11722.

[6] Chen R, Ding Z, Zheng S, et al. MAGIS: Memory Optimization via Coordinated Graph Transformation and Scheduling for DNN[A]. In: Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3[C]. New York: ACM, 2024: 607-621. DOI: 10.1145/3620666.3651330.

[7] Niu W, Guan J, Wang Y, et al. DNNFusion: Accelerating Deep Neural Networks Execution with Advanced Operator Fusion[A]. In: Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation[C]. New York: ACM, 2021: 883-898. DOI: 10.1145/3453483.3454083.

[8] Mei L, Goetschalckx K, Symons A, et al. DeFiNES: Enabling Fast Exploration of the Depth-First Scheduling Space for DNN Accelerators through Analytical Modeling[A]. In: 2023 IEEE International Symposium on High-Performance Computer Architecture[C]. Piscataway: IEEE, 2023: 570-583. DOI: 10.1109/HPCA56546.2023.10071098.

[9] Jacob B, Kligys S, Chen B, et al. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference[A]. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition[C]. Piscataway: IEEE, 2018: 2704-2713. DOI: 10.1109/CVPR.2018.00286.